

Camada de Acesso a Dados

Eduardo Piveta

Introdução

- É interessante dividir um sistema em camadas, de forma que cada uma delas se preocupe com diferentes interesses de uma aplicação.
- Por exemplo, no aplicativo de conexão ao banco de dados, o código da interface com o usuário (console) está misturado com código de acesso a dados.

Introdução (2)

- Esta aula visa mostrar como separar o código de acesso a dados da interface, de forma que ele possa ser reusado para outros tipos de interfaces (desktop, web, dispositivos móveis).
- Na próxima aula como tornar esta camada mais genérica...
- ...evitando tantas repetições.

Exemplo - Interface

```
public class DBExemploConexao {
    public static void main(String[] args) throws ClassNotFoundException, SQLException {
        Class.forName("org.postgresql.Driver");
        Connection c = DriverManager.getConnection(
            "jdbc:postgresql://localhost/livraria",
            "postgres",
            "aluno");

        Statement s = c.createStatement();
        System.out.println("Listando todos os clientes...");
        ResultSet rs = s.executeQuery("SELECT * FROM clientes");
        while (rs.next())
            System.out.println(rs.getInt("codigo") + " - " + rs.getString("nome"));

        System.out.println("Apagando todos os clientes...");
        s.executeUpdate("DELETE FROM clientes");

        System.out.println("Listando todos os clientes novamente...");
        rs = s.executeQuery("SELECT * FROM clientes");
        while (rs.next())
            System.out.println(rs.getInt("codigo") + " - " + rs.getString("nome"));

        System.out.println("Inserindo clientes ...");
        PreparedStatement p = c.prepareStatement(
            "INSERT INTO clientes (codigo, nome) VALUES (?, ?)");
        p.setInt(1, 1);
        p.setString(2, "Fulano");
        p.executeUpdate();
        p.setInt(1, 2);
        p.setString(2, "Baltazano");
        p.executeUpdate();
        p.setInt(1, 3);
        p.setString(2, "Cicliano");
        p.executeUpdate();

        System.out.println("Listando todos os clientes novamente...");
        rs = s.executeQuery("SELECT * FROM clientes");
        while (rs.next())
            System.out.println(rs.getInt("codigo") + " - " + rs.getString("nome"));

        System.out.println("Alterando o cliente 1...");
        p = c.prepareStatement(
            "UPDATE clientes SET nome = ? WHERE codigo = 1");
        p.setString(1, "Fulano da Tel");
        p.executeUpdate();

        System.out.println("Listando todos os clientes novamente...");
        rs = s.executeQuery("SELECT * FROM clientes");
        while (rs.next())
            System.out.println(rs.getInt("codigo") + " - " + rs.getString("nome"));

        rs.close();
        p.close();
        c.close();
    }
}
```

```
public class DBExemploConexao {
    public static void main(String[] args) {

        System.out.println("Listando todos os clientes...");

        while (rs.next())
            System.out.println(rs.getInt("codigo") + " - " + rs.getString("nome"));

        System.out.println("Apagando todos os clientes...");

        System.out.println("Listando todos os clientes novamente...");

        while (rs.next())
            System.out.println(rs.getInt("codigo") + " - " + rs.getString("nome"));

        System.out.println("Inserindo clientes ...");

        System.out.println("Listando todos os clientes novamente...");

        while (rs.next())
            System.out.println(rs.getInt("codigo") + " - " + rs.getString("nome"));

        System.out.println("Alterando o cliente 1...");

        System.out.println("Listando todos os clientes novamente...");

        while (rs.next())
            System.out.println(rs.getInt("codigo") + " - " + rs.getString("nome"));
    }
}
```

Separando interesses com DAOs

- O acesso a dados pode ser separado das camadas superiores (negócios, interface) através de classes de acesso a dados (DAOs).
- Esta classe é usada para separar as camadas superiores da lógica e dos detalhes de acesso as bases de dados
 - Sejam elas em memória, em disco etc.

Exemplo Atualizado – Buscar todos

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class DBExemploConexao {
    public static void main(String[] args) throws ClassNotFoundException, SQLException {
        Class.forName("org.postgresql.Driver");
        Connection c = DriverManager.getConnection(
            "jdbc:postgresql://localhost/livraria",
            "postgres",
            "aluno");

        Statement s = c.createStatement();
        System.out.println("Listando todos os clientes...");
        ResultSet rs = s.executeQuery("SELECT * FROM clientes");
        while (rs.next())
            System.out.println(rs.getInt("codigo") + " - " + rs.getString("nome"));
    }
}
```



```
public class DBExemploConexao {
    public static void main(String[] args) {
        ClientesDAO cs = new ClientesDAO();

        System.out.println("Listando todos os clientes...");
        for (Cliente temp : cs.buscarTodos())
            System.out.println(temp.getCodigo() + " - " + temp.getNome());
    }
}
```

Apagando clientes

```
System.out.println("Apagando todos os clientes...");  
s.executeUpdate("DELETE FROM clientes");
```



```
System.out.println("Apagando todos os clientes...");  
for (Cliente temp : cs.buscarTodos())  
    cs.excluir(temp);
```

NOTA: No exemplo anterior existia um “excluir todos” que normalmente não é usado na prática. É mais comum precisar apagar um registro individual...

Inserindo clientes

```
System.out.println("Inserindo clientes ...");
PreparedStatement p = c.prepareStatement(
    "INSERT INTO clientes (codigo, nome) VALUES (?, ?)");
p.setInt(1, 1);
p.setString(2, "Fulano");
p.execute();
p.setInt(1, 2);
p.setString(2, "Beltrano");
p.execute();
p.setInt(1, 3);
p.setString(2, "Ciclano");
p.execute();
```



```
System.out.println("Inserindo clientes ...");
Cliente c1 = new Cliente(1, "Fulano");
Cliente c2 = new Cliente(2, "Beltrano");
Cliente c3 = new Cliente(3, "Ciclano");
cs.inserir(c1);
cs.inserir(c2);
cs.inserir(c3);
```


Alterando clientes

```
System.out.println("Alterando o cliente 1...");  
p = c.prepareStatement(  
    "UPDATE clientes SET nome = ? WHERE codigo = 1");  
p.setString(1, "Fulano de Tal");  
p.execute();
```



```
System.out.println("Alterando o cliente 1...");  
c1.setNome("Fulano de Tal");  
cs.alterar(c1);
```

P1: E como ficaram os SQLs e cia?

- Foram todos movidos para a classe de acesso a dados de clientes: **ClientesDAO**.
- São usados objetos da classe **Cliente** para representar clientes individuais e transferir dados de clientes:
 - Da interface para a camada de acesso a dados
 - Da camada de acesso a dados para a interface.
- Uma terceira classe auxiliar foi criada para manter os dados específicos do BD...
 - Poderiam ser usadas propriedades (fora do escopo)

P2: Mas isso não cria mais classes?

- Sim, cria, mas a vida é assim. Não dá pra ganhar todas.
- Os atributos de qualidade de um sistema normalmente são conflitantes:
 - Flexibilidade vs. Complexidade
 - Segurança vs. Desempenho
 - Mais Recursos de Linguagem vs. Legibilidade
 - Qualidade vs. Tempo e Custo
 - Manutenibilidade vs. Facilidade de Escrita

P3: Bom, então como ficaram as tais classes para acessar o BD?

```
public class ClientesDAO {  
    public Connection abrir() {..  
  
    public void inserir(Cliente c) {..  
  
    public void alterar(Cliente c) {..  
  
    public void excluir(Cliente c) {..  
  
    public Cliente buscar(Cliente c) {..  
  
    public Collection<Cliente> buscarTodos() {..  
}
```

```
public class Cliente {
    private Integer codigo;
    private String nome;
    public Cliente(int codigo, String nome) {
        setCodigo(codigo);
        setNome(nome);
    }
    public Integer getCodigo() {
        return codigo;
    }
}
```

● ● ●

```
public class BD {  
    public static final String JDBC_DRIVER = "org.postgresql.Driver";  
    static final String SENHA = "aluno";  
    static final String USUARIO = "postgres";  
    static final String URL_CONEXAO = "jdbc:postgresql://localhost/livraria";  
}
```

ClientesDAO – Abrir conexão

- Notem que as exceções ainda não estão sendo tratadas (algumas aulas a frente)...

```
public Connection abrir() {  
    Connection c = null;  
    try {  
        Class.forName(BD.JDBC_DRIVER);  
    } catch (ClassNotFoundException e) {  
        System.out.println("Nao encontrou o driver chamado " + BD.JDBC_DRIVER + " na memoria");  
    }  
    try {  
        c = DriverManager.getConnection(BD.URL_CONEXAO, BD.USUARIO, BD.SENHA);  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
    return c;  
}
```

ClientesDAO – Inserir

```
public void inserir(Cliente c) {  
    Connection conexao = abrir();  
    try {  
        PreparedStatement ps = conexao.prepareStatement(  
            "INSERT INTO clientes (codigo, nome) VALUES (?, ?)");  
        ps.setInt(1, c.getCodigo());  
        ps.setString(2, c.getNome());  
        ps.execute();  
        ps.close();  
        conexao.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

ClientesDAO – Alterar

```
public void alterar(Cliente c) {  
    Connection conexao = abrir();  
    try {  
        PreparedStatement ps = conexao.prepareStatement(  
            "UPDATE clientes SET nome = ? WHERE codigo = ?");  
        ps.setString(1, c.getNome());  
        ps.setInt(2, c.getCodigo());  
        ps.execute();  
        ps.close();  
        conexao.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

ClientesDAO – Excluir

```
public void excluir(Cliente c) {  
    Connection conexao = abrir();  
    try {  
        PreparedStatement ps = conexao.prepareStatement(  
            "DELETE FROM clientes WHERE codigo = ?");  
        ps.setInt(1, c.getCodigo());  
        ps.execute();  
        ps.close();  
        conexao.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```


ClientesDAO – Buscar Todos

```
public Collection<Cliente> buscarTodos() {  
    Connection conexao = abrir();  
    Collection<Cliente> clientes = new ArrayList<Cliente>();  
    try {  
        Statement s = conexao.createStatement();  
        ResultSet rs = s.executeQuery("SELECT * FROM clientes");  
        while (rs.next()) {  
            Cliente temp = new Cliente();  
            temp.setCodigo(rs.getInt("codigo"));  
            temp.setNome(rs.getString("nome"));  
            clientes.add(temp);  
        }  
        rs.close();  
        conexao.close();  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
    return clientes;  
}
```

Exercício

- Implementar as classes de exemplo:
 - DBExemploConexao
 - ClientesDAO
 - Cliente
 - BD
- Listar os registros da tabela após cada operação. Ex.:
 - Listar todos os clientes
 - Apagar todos os clientes
 - Listar todos os clientes novamente
 - Inserir clientes
 - Listar todos os clientes novamente
 - Alterar clientes
 - Listar todos os clientes novamente
- Comparar com DBTesteCliente.zip do site...